

Licenciatura em Engenharia Informática e de Computadores

Dado Eletrónico (*Projeto Final*)

Trabalho realizado por:

Nome: Dinis Melo N° 51500

Nome: Bernardo Lopes N° 51303

Docente: Ezequiel Conde

Arquitetura de Computadores
2024 / 2025 verão

junho de 2025

Índice

1	OBJETIVO	1
2	ANÁLISE DO PROBLEMA	1
3	SOLUÇÃO DESENVOLVIDA	3
3.1	Implementação de Hardware	3
3.2	Mapa de Endereçamento	4
3.3	Organização do Software	6
4	TESTES E RESULTADOS	7
5	RESPOSTAS ÀS QUESTÕES DO ENUNCIADO.....	8
5.1	Explicação da solução adotada para ligar o circuito <i>FED</i> à placa <i>SDP16</i>.....	8
5.2	Cálculos realizados para determinar as temporizações envolvidas no trabalho	8
5.3	Latência máxima do sistema no atendimento dos pedidos de interrupção gerados pelo circuito <i>FED</i>	9
6	CONCLUSÕES	10
7	REFERÊNCIAS	11
8	ANEXO A – CÓDIGO FONTE DO TRABALHO	12

1 Objetivo

Este trabalho tem como objetivo principal analisar e compreender o funcionamento do *hardware* que rodeia um processador no contexto do desenvolvimento de programas em linguagem *assembly*. Serão abordados os seguintes temas: operações de entrada e saída de dados, controlo de temporização, gestão de interrupções externas e a estruturação dos programas através da utilização de rotinas.

2 Análise do Problema

O projeto desenvolvido consistiu na criação de um sistema embebido baseado no processador P16, com o objetivo de simular o funcionamento de um dado eletrónico. Este sistema permite gerar, de forma pseudoaleatória, números inteiros positivos entre 1 e 9, dependendo do tipo de dado selecionado. Foram considerados dados equilibrados com 4, 6, 8 e 12 faces. O sistema utiliza como entradas os sinais provenientes do DIP-switch da placa SDP16. O sinal ROLL, ligado ao interruptor 0, é utilizado para simular o lançamento do dado através de uma transição descendente, ou seja, da posição OFF para ON. Na figura 1, está representado o sistema.

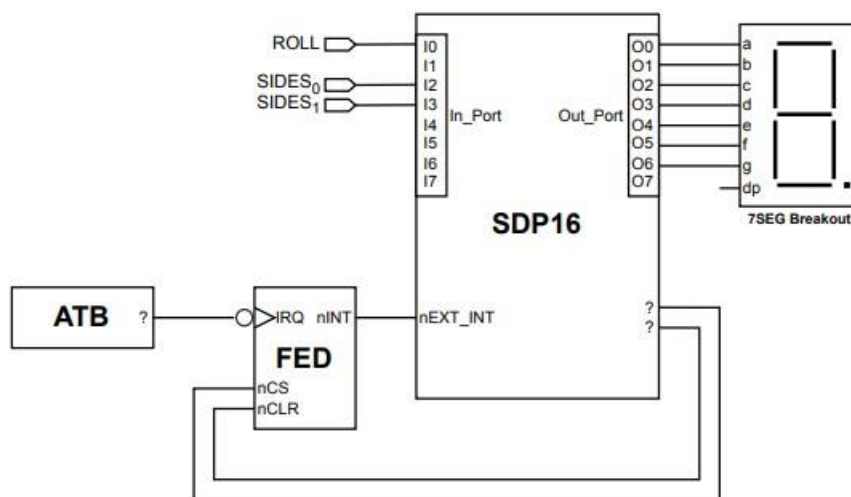


Figura 1: Diagrama de blocos do protótipo do sistema.

Já os sinais SIDES, definidos pelos interruptores 2 e 3, determinam o número de faces do dado a ser simulado no momento do lançamento. A combinação dos dois bits de SIDES permite selecionar entre quatro tipos de dados: com 4, 6, 8 ou 12 faces, sendo que este último é simulado como dois dados de 6 faces, apresentando apenas valores compatíveis com essa configuração.

À saída, o sistema controla os segmentos do mostrador de 7 segmentos através das linhas do porto de saída da SDP16, sendo estes utilizados tanto para apresentar o efeito visual do lançamento como para mostrar o número gerado. Ao iniciar, o sistema mantém o mostrador ligado com um valor correspondente a uma das faces possíveis do dado selecionado. Esse estado inicial permanece até que o jogador acione o

interruptor correspondente ao sinal ROLL, o que provoca a transição necessária para desencadear o lançamento. Esta transição também serve para capturar o estado atual de SIDES, determinando o tipo de dado a lançar.

Para simular o movimento do dado físico, o sistema gera um efeito luminoso no mostrador durante, no mínimo, um segundo. Concluído esse efeito visual, o sistema apresenta o número correspondente ao lançamento, que é gerado através de um mecanismo pseudoaleatório. A geração do número pseudoaleatório é realizada através de uma rotina baseada num gerador congruencial linear (*Linear Congruential Generator* – LCG). Este método utiliza uma variável global denominada *seed*, a qual é atualizada em cada chamada à rotina *rand*. A função não recebe argumentos e devolve um número de 16 bits, calculado a partir da evolução da semente conforme a expressão clássica do LCG: $seed = (a \times seed + c) \bmod m$. No caso implementado, os multiplicadores e incrementos utilizados foram os valores 214013 e 2531011, respetivamente, adequados para gerar uma boa dispersão de valores dentro do intervalo possível.

Internamente, a *seed* é representada em 32 bits, dividida em duas palavras de 16 bits. A rotina efetua a multiplicação e a adição necessárias de forma manual, usando instruções adequadas para operações em múltiplos registos e garantindo a preservação dos resultados intermédios nos registos temporários. Em seguida, o novo valor da *seed* é armazenado para ser utilizado numa futura chamada. O valor devolvido pela função corresponde aos 16 bits mais significativos do resultado final, garantindo assim uma maior entropia e melhor aleatoriedade.

Caso o valor gerado atinja o valor máximo (*RAND_MAX*), a *seed* é automaticamente reiniciada para zero, evitando estagnação ou repetições. Este mecanismo assegura que o gerador não fica preso num único valor. A seleção final da face do dado é feita a partir deste número gerado, aplicando-se a operação de resto da divisão pelo número de faces do dado atualmente selecionado, o que garante que o valor final apresentado no mostrador está dentro do intervalo correto e é coerente com a configuração de SIDES.

Após a apresentação do resultado, o sistema permanece nesse estado durante um período de 10 segundos, ao fim do qual retorna automaticamente ao estado de espera, pronto para um novo lançamento. Para garantir o cumprimento das restrições temporais associadas ao efeito luminoso e ao intervalo entre lançamentos, é utilizado o circuito FED. Este é responsável pela geração de interrupções periódicas com base num sinal de relógio fornecido pela placa ATB, permitindo implementar com precisão os tempos de espera definidos no enunciado.

3 Solução Desenvolvida

3.1 Implementação de Hardware

A implementação do sistema baseou-se na integração das placas SDP16, ATB e 7SEG Breakout, em conjunto com o circuito FED (Falling Edge Detector), permitindo concretizar um dado eletrónico com temporizações fiáveis e resposta eficiente a eventos externos. O circuito FED foi ligado ao sistema com o propósito de gerar interrupções periódicas, utilizando como fonte de sinal de relógio o oscilador da placa ATB. Estas interrupções são fundamentais para garantir o controlo temporal do sistema, nomeadamente para implementar a animação visual com duração mínima de um segundo e a espera de dez segundos antes de permitir um novo lançamento.

A ligação entre o FED e o processador P16 foi realizada através da entrada de interrupção externa do processador, assegurando que uma transição descendente no sinal ROLL, gerada pela comutação do interruptor 0 do DIP-switch presente na placa SDP16, desencadeia a execução da rotina de interrupção definida no programa. Tanto o sinal ROLL como os sinais SIDES, provenientes dos interruptores 2 e 3 do DIP-switch, são lidos diretamente a partir do porto de entrada da SDP16, enquanto a apresentação do valor sorteado no mostrador de 7 segmentos é efetuada através do porto de saída, ao qual os segmentos do mostrador estão conectados.

O barramento de dados e os sinais de controlo foram configurados para garantir acessos sincronizados entre o processador e os periféricos adicionais, promovendo uma comunicação segura e eficaz. A descodificação de endereços foi concebida de forma a evitar conflitos com os dispositivos já integrados na SDP16, atribuindo ao FED um endereço exclusivo (0xFF40) para os seus registos de controlo e estado. Esta organização garante a separação funcional entre os diferentes componentes do sistema e facilita a sua integração no espaço de endereçamento do processador. A estrutura global do sistema pode ser representada por um diagrama de blocos onde se evidenciam as interligações entre o processador P16, o circuito FED, os portos de entrada e saída, o mostrador 7SEG e a placa ATB, refletindo uma arquitetura modular e clara, adequada à finalidade pedagógica e funcional do projeto desenvolvido.

3.2 Mapa de Endereçamento

O mapa de endereçamento do sistema foi concebido para integrar de forma harmoniosa os dispositivos preexistentes na placa SDP16 com os periféricos adicionais requeridos pelo projeto, nomeadamente o circuito FED. A memória principal, bem como os portos de entrada e saída padrão, já se encontram mapeados em regiões fixas no espaço de endereços do processador P16.

No sistema desenvolvido, o porto de entrada está mapeado no endereço 0xFF80, permitindo a leitura direta dos sinais provenientes do DIP-switch da placa SDP16. Este porto disponibiliza os sinais de controlo SIDES (bits 2 e 3) e ROLL (bit 0), os quais são lidos através da rotina `inport_read`, utilizando a instrução `ldrb r0, [r1, #0]` após o carregamento do endereço com `mov` e `movt`.

O porto de saída, por sua vez, está localizado no endereço 0xFFC0, sendo responsável por controlar os segmentos do mostrador de 7 segmentos. Cada valor escrito neste endereço determina o padrão de segmentos iluminados, conforme os códigos definidos na tabela `display_7seg_codes`. A escrita é efetuada através da rotina `outport_write`, que armazena em `r0` o padrão a apresentar e o envia para o porto com a instrução `strb r0, [r1, #0]`.

O circuito FED foi mapeado no endereço 0xFF40, utilizado para controlar a ativação das interrupções. Este endereço foi escolhido por se encontrar fora da zona reservada à memória e aos portos padrão da SDP16, garantindo a inexistência de sobreposição. A escrita neste endereço é feita na rotina `main` e na rotina de interrupção `isr`, onde se utiliza a combinação das instruções `mov`, `movt` e `strb` para carregar o endereço e enviar o sinal de ativação. O valor escrito neste endereço é irrelevante para a lógica funcional, servindo apenas como pulso de controlo.

A memória de dados é usada para armazenar variáveis e tabelas de apoio ao funcionamento do sistema. As variáveis `seed`, `sum`, `dice_sides_comb` e `display_7seg_codes` estão organizadas na secção `.data` e acedidas por endereçamento indireto. O vetor `seed` ocupa duas palavras (64 bits), correspondentes ao estado atual do gerador pseudoaleatório. A variável `sum`, usada como base de tempo, ocupa um único byte e é incrementada ciclicamente pela rotina de interrupção. O vetor `dice_sides_comb` contém os valores 4, 6, 8 e 12, usados para traduzir os bits SIDES num número de faces do dado. Já o vetor `display_7seg_codes` fornece os padrões de segmentos correspondentes a cada dígito.

A pilha foi alocada na secção `stack`, com um espaço reservado de 64 bytes, terminado pelo símbolo `stack_top`, cujo endereço é carregado no registo `sp` durante a inicialização do sistema.

A distribuição dos endereços evidencia uma separação clara entre as regiões destinadas à entrada, saída, controlo de interrupções e memória de dados. Esta organização garante não apenas a compatibilidade com a arquitetura do processador P16, mas também a extensibilidade do sistema para suportar novos periféricos, caso necessário, sem interferir com os componentes existentes.

Endereço Inicial	Endereço Final	Componente	Tipo de Acesso	Descrição
0x0000	0x7FFF	Memória Principal	Leitura/Escrita	Espaço reservado para instruções, dados, variáveis globais e constantes.
0xFF40	0xFF40	FED (nC _s _ExT1)	Escrita	Ativação do circuito FED para permitir interrupções externas.
0xFF80	0xFF80	Porto de Entrada	Leitura	Leitura dos sinais ROLL (bit 0) e SIDES (bits 2 e 3) do DIP-switch.
0xFFC0	0xFFC0	Porto de Saída	Escrita	Escrita dos códigos dos segmentos do mostrador de 7 segmentos.
—	—	seed (64 bits)	Leitura/Escrita	Estado interno do gerador pseudoaleatório (srand / rand).
—	—	sum (8 bits)	Leitura/Escrita	Contador de base de tempo atualizado via interrupções do FED.
—	—	dice_sides_comb	Leitura	Tabela de tradução dos bits SIDES para o número de faces do dado.
—	—	display_7seg_codes	Leitura	Tabela com os padrões dos dígitos a apresentar no mostrador 7SEG.
Topo da RAM	Topo - 63	Pilha (Stack)	Leitura/Escrita	Espaço reservado para a pilha do sistema (64 bytes), iniciado com sp.

3.3 Organização do Software

A organização do software desenvolvido segue uma abordagem modular, estruturada em rotinas bem definidas que asseguram a separação funcional das tarefas do sistema. A arquitetura do programa foi concebida para responder aos requisitos definidos no enunciado, nomeadamente a geração pseudoaleatória de números, o controlo do mostrador de 7 segmentos, a leitura do estado dos interruptores e a implementação de temporizações com base em interrupções externas.

A execução do programa tem início na rotina `program`, responsável pela definição do ponteiro de pilha através do carregamento do endereço `stack_top` para o registo `sp`. Em seguida, o controlo é transferido para a rotina `main`, onde são ativadas as interrupções externas e inicializado o gerador pseudoaleatório através da chamada à função `srand`, com uma semente fixa. A ativação do circuito FED é realizada por uma escrita no endereço correspondente (`nCs_Ext1`), assegurando que o sistema começa a receber interrupções periódicas.

O ciclo principal do sistema está encapsulado num loop contínuo, implementado na rotina `main`, que coordena as seguintes etapas: espera pela ativação do interruptor ROLL (`roll_reader`), execução de uma animação no mostrador (`display_7seg_animation`), cálculo do resultado aleatório (`obtain_result`) e apresentação do valor final (`final_result`). Após estas operações, a rotina `sleep` é chamada para efetuar uma pausa de 10 segundos antes de retomar o ciclo, utilizando para tal o valor da variável global `sum`, que é incrementada a cada interrupção gerada pelo FED.

A rotina de interrupção (`isr`) é responsável por reativar o FED e incrementar a variável `sum`, que atua como um contador de tempo. Esta operação simples permite que o temporizador funcione em segundo plano, sem bloquear a execução principal do programa.

As rotinas auxiliares incluem a `inport_read` e `outport_write`, que encapsulam respetivamente a leitura e escrita nos portos de entrada e saída, simplificando o acesso aos sinais SIDES, ROLL e ao mostrador 7SEG. A rotina `dice_sides` traduz os bits SIDES num número de faces através de uma tabela de conversão, enquanto `rand` e `umull32` implementam um gerador congruencial linear de números pseudoaleatórios com aritmética de 32 bits. O valor aleatório gerado é reduzido ao intervalo desejado através da função `division`, que realiza uma divisão sem sinal para determinar o resultado final do dado.

A modularidade do programa é evidenciada pela separação entre a lógica principal, as rotinas de manipulação de hardware e os mecanismos de suporte à aleatoriedade e temporização. Esta estrutura facilita a manutenção e a compreensão do sistema, ao mesmo tempo que assegura o cumprimento rigoroso dos requisitos funcionais e temporais definidos no enunciado.

4 Testes e Resultados

A validação do sistema desenvolvido foi realizada em duas fases complementares: inicialmente em simulação, utilizando a ferramenta p16sim, e posteriormente em ambiente experimental real, com recurso à placa SDP16, ao circuito FED, à placa ATB e ao mostrador 7SEG Breakout. Estas abordagens permitiram garantir que o sistema respondia corretamente aos requisitos funcionais e temporais definidos no enunciado do projeto.

Durante a fase de simulação, foi testada a lógica de leitura das entradas (ROLL e SIDES), a execução da rotina de geração pseudoaleatória e a transição entre os diferentes estados do sistema. A simulação permitiu verificar que o valor apresentado no mostrador era coerente com o número de faces do dado selecionado, e que o sistema se mantinha no estado de espera após a apresentação do resultado.

No ambiente experimental, foi confirmada a funcionalidade completa do sistema em condições reais de operação. O mostrador de 7 segmentos reagiu corretamente à transição no sinal ROLL, apresentando um efeito visual durante aproximadamente 1 segundo e, posteriormente, o valor gerado de forma pseudoaleatória. Mediu-se também o intervalo de 10 segundos durante o qual o sistema permaneceu inativo antes de aceitar um novo lançamento, comprovando-se a exatidão das temporizações implementadas através do circuito FED.

As interrupções geradas pelo FED foram devidamente tratadas pelo sistema, com tempos de resposta compatíveis com o funcionamento esperado. Fotografias do mostrador em funcionamento e capturas de ecrã da execução em simulação foram registadas para documentar o processo de validação. É importante destacar que os valores apresentados após os lançamentos mostraram uma distribuição equilibrada e coerente com a seleção feita via SIDES.

Durante os testes, não foram identificados comportamentos inesperados. Eventuais pequenos ajustes no tempo de duração do efeito luminoso foram realizados ajustando o divisor de frequência do sinal de relógio aplicado ao FED. No geral, os resultados demonstraram que o sistema responde corretamente às entradas, executa a geração pseudoaleatória com eficiência e respeita os limites temporais impostos, validando a solução proposta com sucesso.

5 Respostas às Questões do Enunciado

5.1 Explicação da solução adotada para ligar o circuito *FED* à placa *SDP16*

A ligação do circuito FED ao sistema foi realizada através da atribuição de um endereço específico (0xFF40) à linha de ativação nCS do FED. A ativação do FED é feita por escrita nesse endereço, o que permite o seu correto funcionamento. Esta operação ocorre no início do programa, na rotina main, e é repetida no interior da rotina de interrupção (isr), assegurando o rearmar do circuito após cada utilização. A deteção de transições descendentes no sinal ROLL, provenientes do interruptor 0 do DIP-switch, é assim possível através da geração de uma interrupção externa ligada ao FED, que se encontra conectado ao sinal nINT do processador P16.

5.2 Cálculos realizados para determinar as temporizações envolvidas no trabalho

Foram consideradas duas temporizações principais:

a) Efeito luminoso de 1 segundo:

A animação é realizada na rotina `display_7seg_animation`, que executa um ciclo com 8 passos visuais, sendo cada um seguido de um atraso implementado pela rotina `sleep` com valor $r0 = 10$. Cada incremento da variável `sum` corresponde a uma interrupção do FED, gerada a cada 100 ms. Assim, $10 \times 100 \text{ ms} = 1 \text{ s}$, garantindo o tempo mínimo requerido para o efeito.

b) Espera de 10 segundos após o lançamento:

Após apresentar o resultado final, o sistema entra novamente na rotina `sleep` com $r0 = 100$. Como o FED continua a gerar interrupções a cada 100 ms, este valor corresponde a um total de $100 \times 100 \text{ ms} = 10 \text{ s}$, assegurando o período de espera especificado antes de aceitar novo lançamento.

5.3 Latência máxima do sistema no atendimento dos pedidos de interrupção gerados pelo circuito *FED*

A latência máxima corresponde ao tempo decorrido entre o instante em que o circuito FED gera um pedido de interrupção e o momento em que o processador P16 inicia a execução da rotina de interrupção externa (isr).

No presente projeto, o circuito FED foi alimentado com um sinal de relógio externo de 10 Hz proveniente da placa ATB, o que implica a geração de uma interrupção externa a cada 100 ms. Já o processador P16, funciona a 50 MHz, de acordo com práticas habituais em sistemas didáticos semelhantes.

A latência é dominada pelo tempo necessário para concluir a instrução em execução, guardar o contexto (PC, LR, CPSR) e transferir o controlo para o vetor da interrupção. Assumindo um cenário conservador, esse processo pode exigir 5 a 8 ciclos de clock, o que corresponde a um tempo de 100 a 160 ns com uma frequência de 50 MHz (20 ns por ciclo).

Este valor é extremamente inferior ao intervalo entre interrupções (100 ms), garantindo que o sistema pode responder pontualmente a todos os eventos gerados pelo FED. Assim, conclui-se que a latência máxima de atendimento da interrupção é inferior a 200 ns, sendo totalmente compatível com os requisitos do sistema.

5.4 Tempo que demora, no pior caso, a execução da rotina utilizada para o atendimento da interrupção externa

A rotina de interrupção externa (isr) executa um conjunto de operações essenciais ao funcionamento do sistema, incluindo a preservação dos registos de trabalho, a reativação do circuito FED através da escrita no seu endereço de seleção (nCS_EXT1), a leitura e incremento da variável sum, e a posterior restauração do contexto do processador. Esta rotina é composta por catorze instruções, algumas das quais envolvem acessos à memória, como as instruções ldr e strb, que podem ter uma duração superior a um ciclo de clock. Considerando o pior caso, em que esses acessos são mais demorados, estima-se que a rotina consuma até vinte ciclos de clock. Assumindo que o processador P16 opera a uma frequência de 50 MHz, valor típico em sistemas baseados na placa SDP16, cada ciclo corresponde a 20 nanosegundos, o que resulta num tempo total de execução da rotina inferior a 400 nanosegundos. Este tempo é extremamente reduzido face ao intervalo entre interrupções geradas pelo circuito FED, que é de 100 milissegundos (frequência de 10 Hz, aplicada externamente via placa ATB). Assim, conclui-se que a rotina de atendimento da interrupção externa é suficientemente rápida para garantir a resposta atempada e fiável do sistema.

6 Conclusões

Com a realização deste trabalho, foram plenamente atingidos todos os objetivos propostos. Foi desenvolvido com sucesso um protótipo de um sistema embebido, baseado no processador *P16*, capaz de simular o lançamento de um dado eletrónico. O sistema permite a seleção de diferentes tipos de dados, com 4, 6, 8 ou 12 faces, através do sinal *SIDES*, determinado pela posição dos interruptores do *DIP-switch*, e realiza a geração pseudoaleatória do número resultante, que é apresentado no mostrador de 7 segmentos. O lançamento do dado é detetado por uma transição no sinal *ROLL*, também controlado por um interruptor, simulando a interação do utilizador com o sistema.

Deste modo, foram alcançados todos os objetivos inicialmente definidos, incluindo a implementação das operações de entrada e saída de dados com recurso aos interruptores e ao mostrador; a criação de rotinas de temporização, com destaque para o efeito visual que simula o movimento do dado e o tempo de espera entre lançamentos; a utilização de interrupções externas para iniciar o lançamento; e ainda a correta organização do programa em rotinas bem estruturadas, facilitando a sua leitura e manutenção.

Ao longo do desenvolvimento deste projeto, os conhecimentos adquiridos nas aulas foram significativamente aprofundados. Verificou-se uma consolidação teórica dos conceitos de programação em linguagem *assembly* e do funcionamento de sistemas embebidos, aliada à experiência prática da sua aplicação direta em *hardware* real. Este trabalho contribuiu, assim, para um reforço sólido das competências técnicas e para o desenvolvimento de uma maior autonomia na criação e implementação de soluções de baixo nível.

7 Referências

- [1] Dias, Tiago: Sistema de interrupções do P16 – Guia de Laboratório da Atividade Laboratorial #4 v1.8. ISEL, Lisboa, Portugal, maio 2025.
- [2] Paraíso, José e Tiago Dias: Placa de Desenvolvimento SDP16 – Manual de Utilização. ISEL, Lisboa, Portugal, junho 2020.
<https://iselppt.sharepoint.com/:b:/s/acp/Ed9PGY5JKnJEsf3rDV2skuIBzBYv4IBaUSR8Y2Ky0tgT4g?e=pvQggg> (Acedido em 01-06-2025).
- [3] Sampaio, Pedro: 7SEG Breakout - Esquema Elétrico. ISEL – IPL, Lisboa, Portugal, r24.07 edição, julho 2024. <https://iselppt.sharepoint.com/:b:/s/acp/ESYKLxFtRWFOgiC5AsxIrfABC-f2Z2UpvNXb0wvZCH32dg?e=gg9XUS> (Acedido em 01-06-2025).

8 Anexo A – Código Fonte do Trabalho

```
.equ    STACK_SIZE, 64
.equ    nCs_Ext1, 0xff40
.equ    DICE_SIDES_MASK, 0x0c
.equ    INPORT_ADDRESS, 0xFF80
.equ    OUTPORT_ADDRESS, 0xFFC0
.equ    RAND_MAX_L, 0xFFFF
.equ    RAND_MAX_H, 0xFFFF

.text

b        program
b        isr

program:
    ldr    sp, stack_top_addr
    b        main

stack_top_addr:
    .word  stack_top

main:
    push lr
    mov r0, #0x78
    mov r1, #0x01
    bl    srand
    mrs r0, cpsr
    mov r1, #(1 << 4)
    orr r0, r0, r1
    msr cpsr, r0
    mov r1, #nCs_Ext1 & 0xFF
    movt r1, #(nCs_Ext1 >> 8) & 0xFF
    strb r1, [r1,#0]

loop:
    bl    roll_reader
    bl    display_7seg_animation
    bl    obtain_result
    bl    final_result
    mov  r0, #100
```

```
    bl      sleep
    b       loop
```

isr:

```
    push r0
    push r1
    push r2

    mov  r1, #nCcs_Ext1 & 0xFF
    movt r1, #(nCcs_Ext1 >> 8) & 0xFF
    strb r1, [r1, #0]
    ldr  r0, sum_addr
    ldrb r2, [r0]
    add  r2, r2, #1
    strb r2, [r0]

    pop  r2
    pop  r1
    pop  r0
    movs pc, lr
```

sleep:

```
    ldr r1, sum_addr
    ldrb r2, [r1, #0]
```

sleep_loop:

```
    ldrb r3, [r1, #0]
    sub  r3, r3, r2
    cmp  r3, r0
    blo  sleep_loop
    mov  pc, lr
```

display_7seg_animation:

```
    push lr
    push r4
    push r5
    mov  r4, #9
    mov  r5, #1
```

for:

```
    cmp r5, r4
    bhs for_end
    ldr r3, display_7seg_codes_addr
    ldrb r0, [r3,r5]
    bl outport_write
    mov r0, #10
    bl sleep
    add r5, r5, #1
    b for
for_end:
    pop r4
    pop r5
    pop lr
    mov pc, lr

final_result:
    push lr
    ldr r3, display_7seg_codes_addr
    ldrb r3, [r3,r0]
    mov r0, r3
    bl outport_write
    pop lr
    mov pc,lr

display_7seg_codes_addr:
    .word display_7seg_codes

sum_addr:
    .word sum

umul132:
    push r8
    push r7
    push r6
    push r5
    push r4
    asr r4, r3, #15
    mov r5, r4
```



```
mov     r6, #0
mov     r7, #0
umull32_loop:
    mov     r8, #32
    cmp     r7, r8
    bhs     umull32_ret
    mov     r8, #1
    and     r8, r2, r8
    bzc     umull32_else
    mov     r8, #1
    cmp     r6, r8
    bne     umull32_loop_end
    add     r4, r4, r0
    adc     r5, r5, r1
    b     umull32_loop_end
umull32_else:
    mov     r8, #0
    cmp     r6, r8
    bne     umull32_loop_end
    sub     r4, r4, r0
    sbc     r5, r5, r1
umull32_loop_end:
    mov     r8, #1
    and     r6, r2, r8
    asr     r5, r5, #1
    rrx     r4, r4
    rrx     r3, r3
    rrx     r2, r2
    add     r7, r7, #1
    b     umull32_loop
umull32_ret:
    mov     r0, r2
    mov     r1, r3
    pop     r4
    pop     r5
    pop     r6
    pop     r7
    pop     r8
```

```
mov    pc, lr
```

```
srnd:
```

```
ldr    r2, seed_addr_srand  
str    r0, [r2, #0]  
str    r1, [r2, #2]  
mov    pc, lr
```

```
seed_addr_srand:
```

```
.word  seed
```

```
rand:
```

```
push   lr  
ldr    r2, seed_addr_rand  
ldr    r0, [r2, #0]  
ldr    r1, [r2, #2]  
mov    r2, #( 0x43FD >> 0 ) & 0xFF  
movt   r2, #( 0x43FD >> 8 ) & 0xFF  
mov    r3, #( 0x0003 >> 0 ) & 0xFF  
movt   r3, #( 0x0003 >> 8 ) & 0xFF  
bl umull32  
mov    r2, #( 0x9EC3 >> 0 ) & 0xFF  
movt   r2, #( 0x9EC3 >> 8 ) & 0xFF  
mov    r3, #( 0x0026 >> 0 ) & 0xFF  
movt   r3, #( 0x0026 >> 8 ) & 0xFF  
add    r0, r0, r2  
adc    r1, r1, r3  
mov    r2, #( RAND_MAX_L >> 0 ) & 0xFF  
movt   r2, #( RAND_MAX_L >> 8 ) & 0xFF  
cmp    r0, r2  
bne    rand_save_seed  
mov    r3, #( RAND_MAX_H >> 0 ) & 0xFF  
movt   r3, #( RAND_MAX_H >> 8 ) & 0xFF  
cmp    r1, r3  
bne    rand_save_seed  
mov    r0, #0  
mov    r1, #0
```

```
rand_save_seed:
```

```
ldr    r2, seed_addr_rand
str    r0, [r2, #0]
str    r1, [r2, #2]
mov    r0, r1
pop    pc
```

```
seed_addr_rand:
    .word seed
```

```
obtain_result:
    push lr
    push r7
    bl   dice_sides
    mov  r7, r0
    bl   rand
    mov  r1, r7
    bl   division
    add  r0, r0, #1
    pop  r7
    pop  lr
    mov  pc, lr
```

```
dice_sides:
    push lr
    bl   inport_read
    mov  r2, #DICE_SIDES_MASK
    and  r0, r0, r2
    lsr  r0, r0, #2
    ldr  r1, dice_sides_comb_addr
    ldrb r0, [r1, r0]
    pop  lr
    mov  pc, lr
```

```
dice_sides_comb_addr:
    .word dice_sides_comb
```

```
roll_reader:
    push lr
```

```
    push r4
    push r5
up_wait:
    bl inport_read
    mov r5, #1
    and r4, r0, r5
    mov r5, #0
    cmp r4, r5
    beq up_wait
down_wait:
    bl inport_read
    mov r5, #1
    and r4, r0, r5
    cmp r4, r5
    beq down_wait
    mov r0, #1
    pop r5
    pop r4
    pop lr
    mov pc, lr

division:
    push lr
    push r4
    push r3
    mov     r3, #0
    mov     r4, #0
    mov     r2, #16
div_while:
    lsl     r0, r0, #1
    adc     r3, r3, r3
    lsl     r4, r4, #1
    cmp     r3, r1
    blo     div_if_end
    sub     r3, r3, r1
    add     r4, r4, #1
div_if_end:
    sub     r2, r2, #1
```

```
bne      div_while
mov      r0, r3
pop      r3
pop      r4
pop      lr
mov      pc, lr
```

inport_read:

```
mov      r1, #INPORT_ADDRESS & 0xFF
movt     r1, #(INPORT_ADDRESS >> 8) & 0xFF
ldrb     r0, [r1, #0]
mov      pc, lr
```

outport_write:

```
mov      r1, #OUTPORT_ADDRESS & 0xFF
movt     r1, #(OUTPORT_ADDRESS >> 8) & 0xFF
strb     r0, [r1, #0]
mov      pc, lr
```

.data

seed:

```
.word 1
.word 0
```

dice_sides_comb:

```
.byte 0x04, 0x06, 0x08, 0x0C
```

display_7seg_codes:

```
.byte 0, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F, 0x77, 0x7C, 0x39, 0
```

sum:

```
.byte 0
```

.stack

```
.space  STACK_SIZE
```

stack_top: